Hacettepe University — Computer Engineering
CMP 720 EMBEDDED SYSTEMS DESIGN

Spring 2026 — Mini In-Class Project Series Part 5 - SAMPLE SOLUTION

Lab 5: Automated Testing & Validation

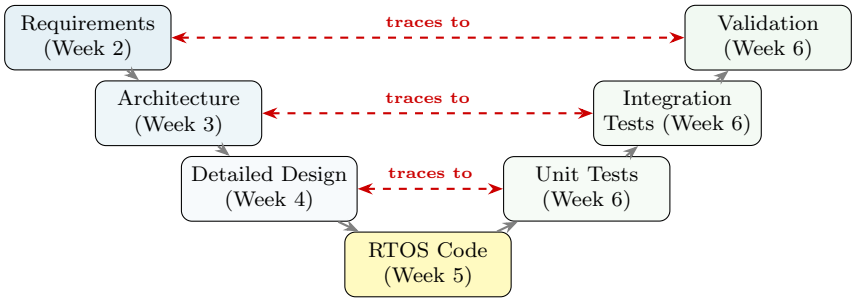
SafePulse — Embedded Patient Monitoring System

Week 6: Evaluation, Validation & Testing

Duration:	1 – 1.5 hours
Tools Required:	Renode, Robot Framework (`renode-test`), terminal, Week 2 spec
Deliverable:	Test suite mapping to Week 2 requirements, all tests passing
Prerequisites:	Completed Lab 4 (Week 5) — working Zephyr RTOS firmware

1 Overview: Closing the V-Model

This is the final lab in the SafePulse series. You will write **automated tests** that validate your firmware against the requirements you specified in Week 2. This closes the V-model: specifications on the left, implementation in the middle, and verification on the right.



> **Why test in simulation?** Renode's Robot Framework integration lets you inject exact sensor values, control virtual time precisely, and observe GPIO state—things that are difficult or impossible on physical hardware. Every test is deterministic and reproducible: no flaky sensor noise, no timing jitter, no "works on my board" problems.

1.1 Learning Objectives

1. Write automated tests using Robot Framework that map directly to Week 2 requirements.
2. Use Renode's test infrastructure to inject sensor values and observe system behavior.
3. Test edge cases and boundary conditions (exact thresholds, fault injection).
4. Understand the relationship between requirements traceability and test coverage.
5. Reason about what *cannot* be tested in simulation vs. physical hardware.

2 Robot Framework + Renode Primer

Robot Framework is a keyword-driven test automation framework. Renode integrates with it via `renode-test`, providing keywords for controlling the simulation. For more details, see the official Renode testing documentation: renode.readthedocs.io/en/latest/introduction/testing.html. Simple example of how to use Renode together with Robot Framework to emulate and test firmware on a host PC without the target hardware: github.com/prdktntwcklr/renode-example.

2.1 Key Renode Test Keywords

Keyword	What It Does
`Execute Command`	Run a Renode monitor command (e.g., write to memory, read registers)
`Start Emulation`	Begin the simulation
`Create Terminal Tester`	Attach to a UART to watch output
`Wait For Line On Uart`	Block until a specific string appears on UART
`Reset Emulation`	Reset the entire emulation state (used between tests)

Caution: `emulation RunFor "N"` (advance virtual time by N seconds) can **only** be used when emulation is **not** running. Once you call `Start Emulation`, use `Wait For Line On Uart` with timeouts to let virtual time advance naturally.

2.2 Test File Structure

Listing 1: Robot Framework test structure for Renode

```
*** Settings ***
# The RENODEKEYWORDS variable (initialized by the renode-test script)
# contains the path to the renode-keywords.robot script
# responsible for setting up the connection with Renode.

Resource        ${RENODEKEYWORDS}
Library         String
Library         Collections
Suite Setup     Setup
Suite Teardown  Teardown
Test Setup      SafePulse Test Setup
Test Teardown   Test Teardown

*** Variables ***
${UART}         sysbus.usart2

*** Keywords ***
SafePulse Test Setup
    Reset Emulation
    Execute Command             mach create "SafePulse"
    Execute Command             machine LoadPlatformDescription @platforms/boards/
        stm32f4_discovery.repl
    Execute Command             machine LoadPlatformDescriptionFromString "vitalsSensor:
         Mocks.DummyI2CSlave @ i2c1 0x55"
    Execute Command             sysbus.cpu PerformanceInMips 168
    Execute Command             sysbus LoadELF @${CURDIR}/build/zephyr/zephyr.elf
    Execute Command             include @${CURDIR}/vitals_sensor.py
    Execute Command             logLevel -1 sysbus.usart2

Set Sensor Values
```

```
    [Documentation]             Inject specific values into the sensor model.
    [Arguments]                 ${hr}    ${spo2}    ${temp_int}    ${temp_frac}
    Execute Command             python "sensor_state['heart_rate'] = ${hr}"
    Execute Command             python "sensor_state['spo2'] = ${spo2}"
    Execute Command             python "sensor_state['temp_int'] = ${temp_int}"
    Execute Command             python "sensor_state['temp_frac'] = ${temp_frac}"

Enable Scenario
    [Documentation]             Enable the built-in deterioration scenario
    Execute Command             python "sensor_state['scenario_active'] = True"
    Execute Command             python "sensor_state['scenario_tick'] = 0"

Check Alarm Pin
    [Documentation]    Read GPIOD_ODR bit 14 (PD14 red LED = alarm output).
    ...                0 = OFF (no alarm), 1 = ON (alarm active)
    [Arguments]        ${expected}
    ${odr}=    Execute Command    sysbus ReadDoubleWord 0x40020C14
    ${odr}=    Strip String    ${odr}
    ${bit}=    Evaluate    (int('${odr}', 0) >> 14) & 1
    Should Be Equal As Integers    ${bit}    ${expected}
    ...    Alarm GPIO PD14: expected=${expected} but got=${bit} (ODR=${odr})

*** Test Cases ***
# ============================================================
# TEST 1: System Boot and Sensor Detection
# Requirement: FR-07 (startup self-test)
# ============================================================
System Should Boot Successfully
    [Documentation]     Verify startup banner and sensor detection
    ${tester}=          Create Terminal Tester    ${UART}
    Set Sensor Values   72    97    36    8
    Start Emulation

    Wait For Line On Uart       SafePulse v0.5          testerId=${tester}
    Wait For Line On Uart       Sensor detected         testerId=${tester}
```

Critical naming rules:

1. **Never** name your custom keywords `Setup`, `Teardown`, `Test Teardown`, or `Create Terminal Tester`—these names are used by Renode's built-in keywords from `renode-keywords.robot`. Shadowing them causes infinite recursion or missing functionality.
2. Use `Test Setup` (not `Suite Setup`) for machine configuration so each test gets a fresh environment via `Reset Emulation`.
3. Sensor values are injected by modifying the Python `sensor_state` dictionary directly—**not** via `sysbus WriteByte`.

3 Requirements Traceability Matrix

Before writing tests, map your Week 2 requirements to test cases. Complete the matrix:

Req ID	Requirement	Test Case	Status
FR-01	Sample vitals every 200 ms	Test 2: Periodic Sampling (indirect)	
FR-02	Alarm within 100 ms when SpO2 < 90%	Test 3: Critical Alarm	
FR-03	Log readings via UART at 1 Hz	Test 2: Periodic Sampling	
FR-04	Deactivate alarm on recovery	Test 5: Alarm Recovery	
FR-05	WARNING state for SpO2 90–94%	Test 4: Warning State	
FR-07	Startup self-test banner	Test 1: Boot Test	
NFR-01	*Alarm latency ≤ 100 ms*	*Test 3 (timing)*	

Fill in the Status column as you complete each test: PASS, FAIL, or SKIP.

4 Implementation Tasks

The provided `safepulse_tests.robot` skeleton contains 6 test cases. Tests 1–2 are complete. You must complete Tests 3–6.

4.1 Test 1: System Boot (Provided)

Purpose: Verify the firmware boots cleanly and confirms sensor presence on UART. This test is complete—verify it passes by running:

```
$ renode-test safepulse_tests.robot
```

4.2 Test 2: Periodic Vital Signs Reporting (Provided)

Test 2 verifies FR-01 and FR-03. We create a terminal tester with `defaultPauseEmulation=true`, set known sensor values, and start the emulation. The tester automatically pauses the instant a UART line matches the vitals regex pattern; we capture that line, resume emulation with a second `Start Emulation`, and capture the next matching line.

Because the Zephyr firmware prefixes every log line with a `k_uptime` timestamp in the format `[NNNN ms]`, we can extract the timestamps from both captured lines with a regular expression and verify that their difference falls within the expected 1000 ± 200 ms window (1 Hz logging in `NORMAL` state).

```
Should Report Vitals Periodically
    [Documentation]     Verify periodic UART output with vital signs.
    ...                 The Zephyr firmware prints with format:
    ...                 [NNNN ms][sample_id] HR=... State=...
    ...                 In NORMAL state the log timer runs at 1 Hz,
    ...                 so consecutive timestamps should differ by
        ~1000ms.

    # Use defaultPauseEmulation so emulation pauses the instant a
        match occurs
    ${tester}=          Create Terminal Tester    ${UART}    timeout
```

```
    =30    defaultPauseEmulation=true
Set Sensor Values    72    97    36    8
Start Emulation

# Wait for first vitals log line - group 1 = timestamp, groups
    2-5 = vitals
${line1}=            Wait For Line On Uart    \\[(\\d+)\\s*ms
    \\].*?HR=(\\d+).*?SpO2=(\\d+).*?Temp=(\\d+\\.?\\d*).*?State
    =(\\w+)  treatAsRegex=true    testerId=${tester}

# Resume emulation and wait for the next vitals log line
Start Emulation
${line2}=            Wait For Line On Uart    \\[(\\d+)\\s*ms
    \\].*?HR=(\\d+).*?SpO2=(\\d+).*?Temp=(\\d+\\.?\\d*).*?State
    =(\\w+)  treatAsRegex=true    testerId=${tester}

# Extract timestamps directly from capture group 1
${has_groups}=       Evaluate    "groups" in $line1
${groups_key}=       Set Variable If    ${has_groups}    groups
      Groups
${ts1}=              Set Variable    ${line1["${groups_key}"][0]}
${ts2}=              Set Variable    ${line2["${groups_key}"][0]}

${delta_ms}=         Evaluate    int('${ts2}') - int('${ts1}')
# In NORMAL state the log timer fires at 1 Hz, so expect ~1000ms
    +/- 200ms
Should Be True    ${delta_ms} >= 800 and ${delta_ms} <= 1200
...    FR-03 violated: log period was ${delta_ms}ms, expected
    ~1000ms
```

4.3 Test 3: Critical SpO2 Alarm (FR-02, NFR-01)

This is the most important test—it verifies the safety-critical alarm. Inject normal values first, let the system boot and stabilize, then inject a critical SpO2 value and verify the alarm fires.

Listing 2: TODO 1: Test 3: Critical SpO2 Alarm

```
# ============================================================
# TEST 3: Critical SpO2 Alarm
# Validates: FR-02, NFR-01
#
# TODO:
#       a) Start with normal values, wait for a log line to
#          confirm the system has processed the new sensor
#          values.
#       b) Pre-condition: alarm GPIO must be LOW in normal state
#       c) Inject critical SpO2
#       d) advance 500ms of virtual time (enough for one
#          200ms sensor cycle + processing, with a margin)
#       e) Assert: the GPIO alarm pin must now be HIGH
# ============================================================
Should Trigger Alarm On Critical SpO2
    [Documentation]     Verify alarm fires when SpO2 drops below 90%
```

```
    # TODO: Write this test

    Fail    Not implemented yet
```

```
Should Trigger Alarm On Critical SpO2
    [Documentation]     Verify GPIO alarm fires when SpO2 drops below
        90%.
    ...                 Tests the lock-free GPIO path with NO UART
       dependency.
    ${tester}=          Create Terminal Tester    ${UART}    timeout
       =15
    Set Sensor Values   72    97    36    8
    Start Emulation

    # Boot sync i only UART dependency, just confirms system is alive
    Wait For Line On Uart       Sensor detected         testerId=${
       tester}

    # Pre-condition: alarm GPIO must be LOW in normal state
    Execute Command    pause
    Check Alarm Pin    0

    # Inject critical SpO2 and advance exactly 500ms of virtual time
    # (enough for one 200ms sensor cycle + processing, with margin)
    Set Sensor Values   72    85    36    8
    Execute Command    emulation RunFor "0:0:0.5"

    # Assertion: GPIO alarm pin must now be HIGH
    Check Alarm Pin    1
```

Key insight: The sensor model (`vitals_sensor.py`) ignores I2C writes to the scenario register—the firmware's attempt to start the deterioration scenario during boot has no effect. This means your test-injected values are never overwritten, and you can simply call `Set Sensor Values` at any point to change what the firmware reads.

4.4 Test 4: Warning State (FR-05)

Verify the system enters WARNING when SpO2 is in the 90–94% range.

Listing 3: TODO 2: Test 4: Warning State

```
# ============================================================
# TEST 4: Warning State Detection
# Validates: FR-05
#
# TODO: Set SpO2 to a warning-level value (90-94) and verify
#       "WARNING" appears on UART.
# ============================================================
Should Enter Warning State On Low SpO2
    [Documentation]     Verify WARNING state when SpO2 is 90-94%
    # TODO: Write this test
```

```
    Fail    Not implemented yet
```

```
Should Enter Warning State On Low SpO2
    [Documentation]     Verify WARNING state when SpO2 is 90-94%
    ${tester}=          Create Terminal Tester    ${UART}    timeout
        =15
    Set Sensor Values   72    93    36    8
    Start Emulation

    Wait For Line On Uart       WARNING             testerId=${tester
        }      timeout=10
```

4.5 Test 5: Alarm Recovery (FR-04)

Verify the alarm clears when vitals return to normal. This test has two phases: trigger the alarm, then inject recovery values.

Listing 4: TODO 3: Test 5: Alarm Recovery

```
# ============================================================
# TEST 5: Alarm Recovery (CHALLENGE)
# Validates: FR-04
#
# TODO: Start with critical values, wait for CRITICAL,
#       then inject normal values and wait for NORMAL.
# ============================================================
Should Clear Alarm On Recovery
    [Documentation]     Verify alarm clears when vitals return to normal
    # TODO: Write this test

    Fail    Not implemented yet
```

```
Should Clear Alarm On Recovery
    [Documentation]     Verify alarm clears when vitals return to
        normal
    ${tester}=          Create Terminal Tester    ${UART}    timeout
        =20
    Set Sensor Values   120    85    37    8
    Start Emulation

    Wait For Line On Uart       CRITICAL            testerId=${tester
        }      timeout=10

    Set Sensor Values   72    97    36    8

    Wait For Line On Uart       NORMAL              testerId=${tester
        }      timeout=15
```

4.6 Test 6: Boundary Condition (Write From Scratch)

Write a complete test that validates the behavior at the exact SpO2 = 90% boundary. Refer to your Week 2 state machine design: does SpO2 = 90% trigger CRITICAL or WARNING? Your test should verify *your* design decision.

Listing 5: TODO 4: Test 6: Boundary Condition

```
# ============================================================
# TEST 6: Edge Case - SpO2 at Exact Threshold
# Validates: Boundary behavior at SpO2 = 90%
#
# TODO: SpO2 = 90 should be WARNING (code uses < 90 for CRITICAL).
#       SpO2 = 89 should trigger CRITICAL. Test both.
# ============================================================
Should Handle SpO2 At Threshold Boundary
    [Documentation]     Test behavior at exact SpO2 = 90% boundary
    # TODO: Write this test

    Fail    Not implemented yet
```

```
Should Handle SpO2 At Threshold Boundary
    [Documentation]     Test behavior at exact SpO2 = 90% boundary
    ${tester}=          Create Terminal Tester    ${UART}    timeout
        =15
    Set Sensor Values   72    90    36    8
    Start Emulation

    Wait For Line On Uart       WARNING             testerId=${tester
        }      timeout=10

    Set Sensor Values   72    89    36    8
    Wait For Line On Uart       CRITICAL            testerId=${tester
        }      timeout=10
```

Design note: This test directly validates the boundary condition in `check_thresholds()`: the code uses `spo2 < SPO2_CRITICAL` (strictly less than 90), so exactly 90 falls into WARNING, not CRITICAL. If a student's Week 2 design said ≤ 90 is CRITICAL, their code and test should both reflect that—the test verifies *their* specification.

4.7 Build and Run

Build (if not already built from the last week) with:
`west build -b stm32f4_disco . --pristine`
Run with: `renode-test safepulse_tests.robot`

5 Bonus Tests (If Time Permits)

If you finish early, write tests for these additional scenarios:

Test 7: High Temperature Alert. Set SpO2 normal (97%) but temperature critical (40.5°C). Verify CRITICAL state triggers from temperature alone.

Test 8: Full Deterioration Scenario. Enable the built-in scenario and verify the complete sequence: NORMAL → CRITICAL → WARNING → NORMAL. Note that recovery passes through WARNING before reaching NORMAL.

```
Should Trigger Alarm On Critical Temperature
    [Documentation]     Verify alarm fires when temperature exceeds
        40.0 C
    ${tester}=          Create Terminal Tester    ${UART}    timeout
        =15
    Set Sensor Values   72    97    40    5
    Start Emulation

    Wait For Line On Uart       CRITICAL            testerId=${tester
        }      timeout=10

# Test 8: Full deterioration scenario
Should Complete Full Deterioration Scenario
    [Documentation]     Run built-in scenario and verify state
        transitions
    ${tester}=          Create Terminal Tester    ${UART}    timeout
        =120
    Enable Scenario
    Start Emulation

    Wait For Line On Uart       NORMAL              testerId=${tester
        }      timeout=15
    Wait For Line On Uart       CRITICAL            testerId=${tester
        }      timeout=45
    Wait For Line On Uart       WARNING             testerId=${tester
        }      timeout=45
    Wait For Line On Uart       NORMAL              testerId=${tester
        }      timeout=60
```

6 Reflection Questions

Q1: Look at your requirements traceability matrix. Are there any requirements from Week 2 that are **not** covered by your test suite? Could they be tested in simulation, or do they require physical hardware?

[Your answer]

Likely uncovered requirements include: NFR-04 (72-hour uptime / no memory leaks)—this requires a long-duration soak test that's impractical in a 1.5-hour lab but possible in simu-

lation with accelerated virtual time. NFR-03 (sampling jitter $\leq$ 10 ms)—testable in Renode by capturing timestamps but not implemented in our basic test suite. FR-06 (I2C NACK detection)—testable by modifying the Python peripheral to simulate disconnection but requires more complex test infrastructure. Physical hardware tests needed for: EMI susceptibility, power consumption, actual I2C bus timing, physical buzzer audibility, and operating temperature range. Simulation can never fully replace hardware validation for environmental and electrical characteristics.

Q2: Your tests use `Wait For Line On Uart` with a timeout. What does it mean if a test passes with 0.5 s to spare vs. 4.9 s to spare (out of 5 s timeout)? Does a "pass" always mean the system is working correctly?

[Your answer]

A test that passes with 4.9 s to spare indicates the system responded almost immediately—the alarm fired within the first few sampling cycles. A test passing with 0.5 s to spare suggests the system took ~4.5 s to respond, which might technically "pass" the test but indicates a serious timing problem (the alarm is many seconds late). A pass does **not** guarantee correctness—it only guarantees the expected string appeared within the timeout. A more rigorous test would measure the *actual* elapsed virtual time between the sensor value injection and the alarm appearance, then assert it's within the 100 ms deadline. Robot Framework supports this with Renode's virtual time querying, but it requires more sophisticated test infrastructure.

Q3: If you were to ship SafePulse as a real medical device under IEC 62304, what additional testing would be required beyond what you did today?

[Your answer]

IEC 62304 requires: **Unit testing** of every function (we only did integration/system-level tests). **Code coverage analysis** (statement, branch, MC/DC coverage depending on safety class). **Regression testing** (full suite run on every code change). **Static analysis** (MISRA C compliance, no undefined behavior). **Fault injection** (what happens when memory corrupts, stack overflows, watchdog triggers). **Hardware-in-the-loop testing** on physical boards. **Clinical validation** with real patient scenarios. **Stress/soak testing** (72+ hours continuous operation). **Electromagnetic compatibility (EMC)** testing. **Usability testing** (can nurses understand the alarm patterns?). Documentation: every test must be traceable to a

requirement, and every requirement must have at least one test. All of this must be reviewed and approved by a regulatory body before the device can be sold.

7 Series Wrap-Up

Congratulations—you have completed the full SafePulse development lifecycle:

Week	Phase	What You Built
2	Specification	Requirements table, state machine, block diagram
3	Platform Bring-Up	UART, GPIO, first Renode simulation
4	Driver & Timing	Timer ISR, sensor driver, 100 ms alarm deadline
5	RTOS Migration	Multi-task Zephyr system, priority inversion experiment
6	Validation	Automated test suite mapped to Week 2 requirements

The professional takeaway: You experienced—in miniature—the complete development process for a safety-critical embedded system: specify before you code, design before you implement, test against your specification, and understand that scheduling and priority decisions are not "just software"—they are safety decisions with life-or-death consequences.